

Real-Time Collaborative System for Team and Task Management

Relatório Técnico de Arquitetura e Implementação

Dany Raimundo (8250047), João Mendes (8250051), and Magson Chostak (8250657)

Escola Superior de Tecnologia e Gestão (ESTG),
Instituto Politécnico do Porto (IPP), Felgueiras, Portugal

Abstract. O presente documento detalha a arquitetura e implementação de um sistema colaborativo em tempo real para Gestão de Equipas e Tarefas. Desenvolvida no âmbito da unidade curricular de Paradigmas Emergentes para o Desenvolvimento Web e Mobile, a solução integra uma aplicação *frontend* em Flutter e um *backend* em C# .NET. A arquitetura baseia-se no paradigma *Clean Architecture*, garantindo uma separação rigorosa de responsabilidades. A comunicação serão feitas através de GraphQL para operações transacionais e métricas complexas, e Web-Sockets para reatividade em tempo real. A adoção de padrões de desenho avançados, como CQRS, Builder (com CRTP), Factory e Strategy, reflete o alinhamento total com os objetivos de avaliação estipulados.

1 Stack Tecnológica e Ferramentas

A seleção da *stack* tecnológica foi guiada pela necessidade de suportar uma arquitetura limpa e escalável e motivada pelo interesse académico. A Tabela 1 detalha as tecnologias adotadas em cada camada do sistema e a respetiva justificação arquitetural.

Table 1. Stack Tecnológica e Ferramentas de Desenvolvimento

Camada / Domínio	Tecnologia	Propósito e Justificação
Frontend (Mobile/Web)	Flutter & Dart	Criação de uma interface multiplataforma fluida.
Backend (Core)	C# .NET	Framework robusta para a implementação da <i>Clean Architecture</i> , CQRS e padrões de desenho orientados a objetos.
Persistência de Dados	SQL & Entity Framework Core	Mapeamento objeto-relacional (ORM), para suportar a gravação das entidades de domínio e a herança polimórfica das tarefas.
API & Dados	GraphQL (HotChocolate)	Substituição do paradigma REST clássico para permitir <i>queries</i> flexíveis e polimórficas de métricas e tarefas.
Tempo Real	WebSockets (SignalR)	Estabelecimento de canais bidirecionais para alertas de presença e atualização instantânea de <i>dashboards</i> .
Mapeamento de Objetos	Mapster	Transcrição eficiente e automática entre <i>DTOs</i> e Entidades de Domínio.
Testes de API	Postman / Insomnia	Validação e demonstração independente das <i>queries</i> e <i>mutations</i> do <i>backend</i> .
Controlo de Versões	GitHub	Gestão do código fonte, integração do docente no repositório e rastreabilidade de <i>Story Points</i> .

2 Arquitetura do Sistema: Clean Architecture

O projeto foi estruturado em quatro camadas independentes, promovendo o encapsulamento, a testabilidade e a escalabilidade do sistema:

- **Domain Layer (Core):** O coração do sistema. Contém as regras de negócio puras e as entidades principais (como Utilizadores, Projetos e Tarefas). É aqui que se centralizam os mecanismos estruturais para garantir que os dados assumem sempre um estado válido e seguro na sua instanciação, recorrendo a *Builders* tipados com CRTP para assegurar a reutilização de código e o estrito cumprimento do princípio DRY.

- **Application Layer:** Responsável pela orquestração dos casos de uso através do padrão CQRS. Define *Commands* e *Handlers*, as *Factories* polimórficas e o contrato das estratégias de notificação. Define também as interfaces dos repositórios.
- **Infrastructure Layer:** É a camada encarregue de lidar com os detalhes físicos e de infraestrutura. Para a persistência de dados, utiliza o **Entity Framework Core (EF Core)** como ORM (*Object-Relational Mapper*), garantindo a tradução fluida e segura das entidades de domínio para a base de dados relacional. É também nesta camada que residem os repositórios concretos e os serviços de entrega de mensagens (como os *WebSockets*).
- **Presentation Layer:** Expõe a API GraphQL. Gere a receção de pedidos transcrevendo os *Input DTOs* em *Commands* (via Mapster), e expõe as subscrições de *WebSockets* para o cliente Flutter.

3 Objetivos do Projeto

O objetivo central deste projeto consiste na conceção e desenvolvimento de um sistema colaborativo, a operar em tempo real, dedicado à gestão integrada de projetos, acompanhamento de tarefas e reporte detalhado de horas de trabalho.

A solução foi desenhada para dar resposta às necessidades operacionais e de sincronização de equipas, focando-se nas seguintes vertentes funcionais:

- **Gestão de Projetos e Tarefas:** Permitir a criação, organização e monitorização do estado de vários projetos e das respetivas tarefas, garantindo uma visão clara sobre a que atividades cada membro da equipa está atribuído.
- **Reporte de Horas (Timesheets):** Facilitar o registo rigoroso do tempo investido pelos utilizadores nas diferentes tarefas e categorias de projeto, permitindo cruzar as horas reportadas com o orçamento de tempo (*budget*) estipulado.
- **Presença e Colaboração em Tempo Real:** Proporcionar um ambiente interativo onde é possível identificar, de forma instantânea, o contexto de cada membro da equipa. O sistema permite distinguir em tempo real se um utilizador está ativamente a reportar horas numa tarefa específica ou se está apenas a visualizar os detalhes de um projeto naquele momento.
- **Monitorização de Esforço e Atribuições:** Oferecer à equipa e aos gestores uma visão global sobre a distribuição do trabalho, permitindo saber exatamente onde (em que tarefas) as horas estão a ser gastas, por quem foram reportadas e qual a taxa de progresso das atividades atribuídas.

4 Modelo de Domínio e Polimorfismo

O sistema foi modelado em torno de duas hierarquias principais de herança, permitindo um tratamento polimórfico eficiente tanto na alocação de tempo como na gestão operacional.

4.1 Hierarquia de Gestão de Projetos (ProjectBase)

A classe abstrata `ProjectBase` unifica todas as entidades que requerem alocação temporal e esforço da equipa.

Table 2. Mapeamento da Hierarquia de Gestão de Tempo

Classe	Propriedades Relevantes	Responsabilidade no Domínio
ProjectBase (Abstract)	Id, Title, StartDate, EndDate	Base abstrata que partilha as propriedades temporais obrigatórias.
Project	BudgetHours, ManagerId, TeamId	Representa trabalho faturável ou de desenvolvimento core gerido por um GP.
Holiday	Type (Fixed/Optional)	Entidade para reporte de dias de ausência oficial.
Training	CourseName, Hours	Entidade para tracking de desenvolvimento profissional.

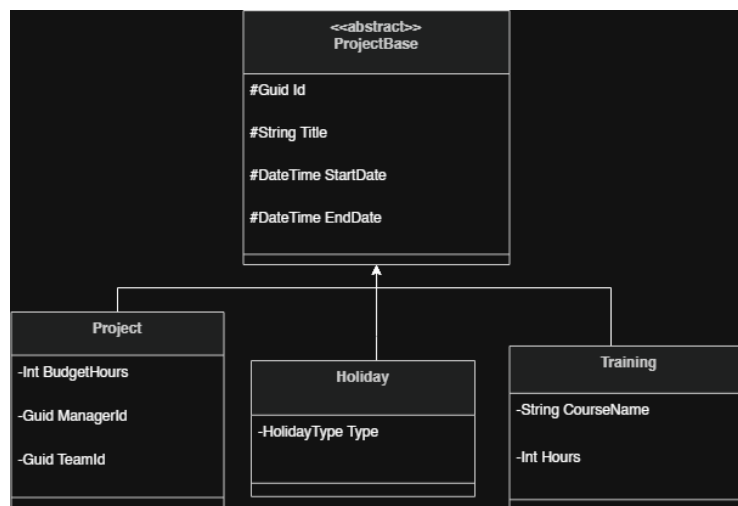


Fig. 1. Diagrama UML correspondente a hierarquia da classe de projetos

4.2 Hierarquia de Gestão de Tarefas (TaskBase)

As tarefas operacionais herdam de `TaskBase`, permitindo métricas consolidadas de desempenho de equipa.

Table 3. Mapeamento da Hierarquia de Gestão de Tarefas

Classe	Propriedades Relevantes	Responsabilidade no Domínio
TaskBase (Abstract)	Id, Title, Status, ProjectId	Base comum; encapsula métodos de negócio como <code>MarkAsDone()</code> .
FeatureTask	StoryPoints	Unidade de desenvolvimento cujo peso é medido para cálculo de velocidade.
BugTask	Severity, Environment	Unidade de correção prioritária associada a um ambiente específico.

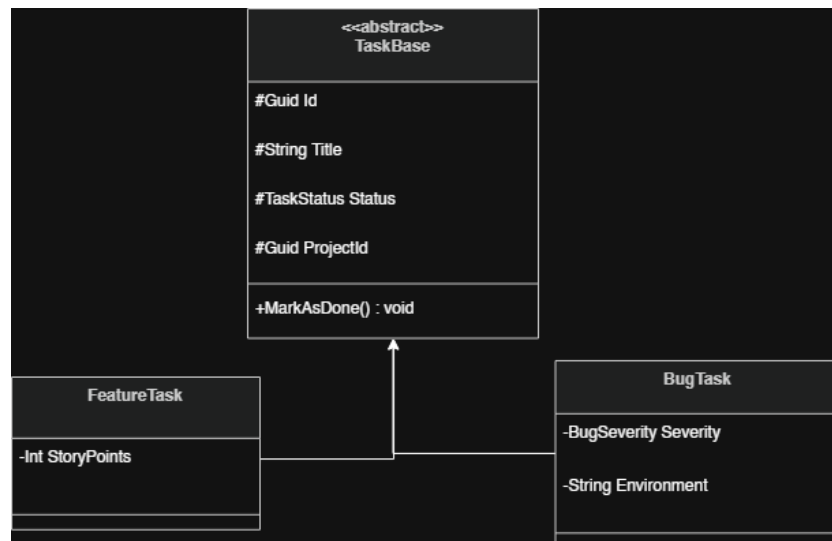


Fig. 2. Diagrama UML correspondente a hierarquia da classe de tarefas

5 Modelo de Entidades e Domínio

O modelo de dados foi desenhado para representar os requisitos de negócio, para garantirmos a integridade relacional da informação. A arquitetura assenta nas seguintes entidades e relações principais:

- **Estrutura Organizacional (User e Team):** Os utilizadores (*Users*) assumem papéis específicos (*Roles*) e encontram-se agrupados em equipas (*Teams*).
- **Núcleo de Gestão (Project):** Atua como a entidade agregadora do trabalho. Cada projeto possui um orçamento de horas (*BudgetHours*), períodos temporais (*StartDate* e *EndDate*), é gerido por um responsável (*Manager*) e encontra-se atribuído a uma equipa.
- **Polimorfismo de Tarefas (FeatureTask e BugTask):** A modelação reflete a natureza distinta das atividades de engenharia. As tarefas associam-se a projetos, mas dividem-se logicamente em: novas funcionalidades (*FeatureTasks*, estimadas através de *Story Points*) e correções de anomalias (*BugTasks*, que exigem o registo da severidade e do ambiente afetado).
- **Sistema de Alertas (Notification):** Entidade dedicada ao registo persistente de eventos e mensagens direcionadas aos utilizadores, para servir de histórico para as notificações em tempo real ou email.

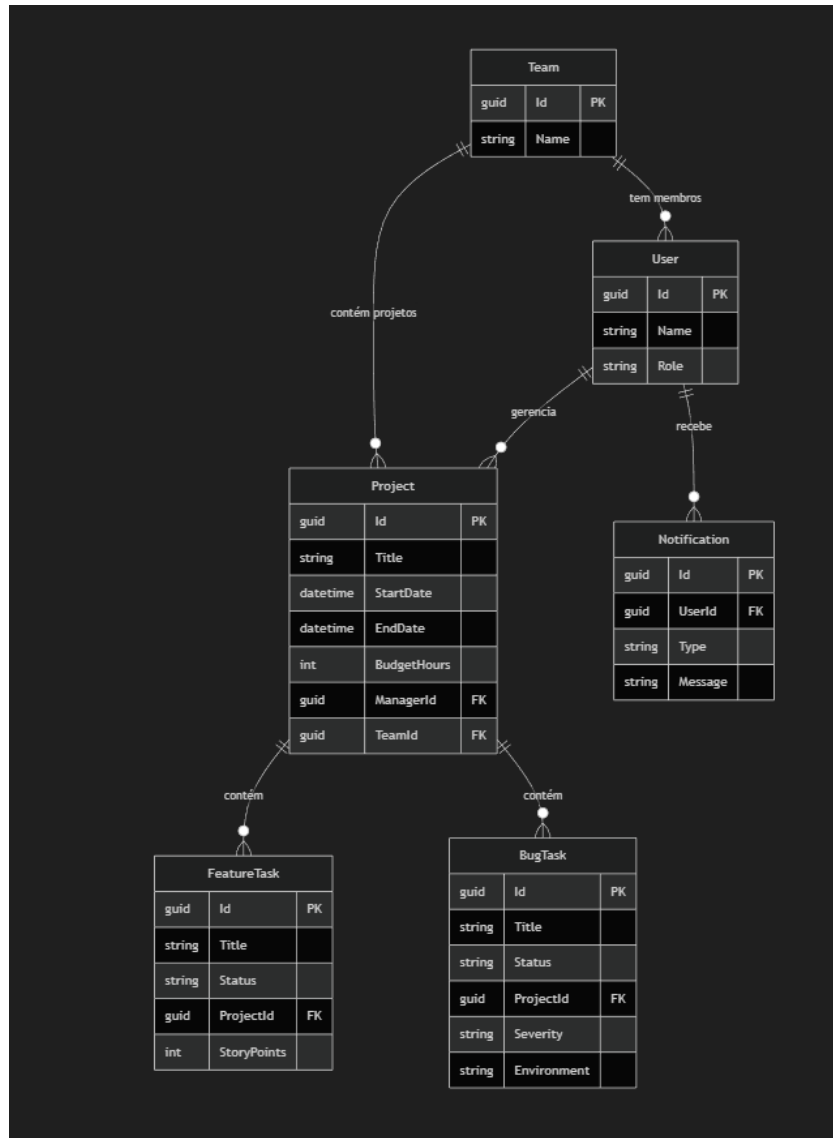


Fig. 3. Modelo de entidades

6 Justificação dos Padrões de Desenho Aplicados

A solução implementa múltiplos *Design Patterns* para resolver problemas estruturais complexos, garantindo que o sistema se mantém eficiente, coeso e altamente escalável:

- **Builder Pattern com CRTP (Domain Layer):** Utilizado para evitar construtores complexos com múltiplos parâmetros e garantir as regras de negócio durante a criação de objetos. A adoção do padrão CRTP (*Curiously Recurring Template Pattern*) permitiu partilhar a lógica de construção entre diferentes tipos de entidades de tempo e tarefas, eliminando a duplicação de código de forma fluente.
- **Factory Pattern (Application Layer):** Atua como o motor central de decisão polimórfica. Ao receber um pedido do exterior, a fábrica avalia o tipo de entidade pretendida e invoca o *Builder* correspondente de forma automática. Isto liberta a interface de utilizador e a API de conterem lógicas condicionais complexas.
- **Strategy Pattern (Infrastructure/Application):** Aplicado ao sistema de envio de notificações. Permite que a aplicação de enviar notificações por, comunicação em tempo real via WebSockets ou envio de e-mail.
- **Singleton Pattern (Infrastructure / Cross-Cutting):** Utilizado para a gestão centralizada do registo de eventos do sistema (*Logs*). Garante a existência de uma instância única global, o que previne conflitos e condições de corrida no acesso aos ficheiros de registo, minimizando também o consumo de memória durante a execução.
- **CQRS (Command Query Responsibility Segregation):** Adotado para separar explicitamente as operações de escrita e alteração de estado das operações exclusivas de leitura de dados. Esta divisão otimiza o desempenho das consultas para os *dashboards* e espelha perfeitamente a natureza bidirecional da API GraphQL.

7 Tecnologias Core e Integração

O sistema foi concebido em torno das duas tecnologias exigidas:

7.1 GraphQL (Data Operations & Mapping)

Utilizado para expor uma API estritamente tipada, cuja implementação no *backend* .NET é suportada pela *framework* **HotChocolate**. O cliente Flutter submete *Input DTOs*, que são automaticamente mapeados para *Commands* internos ao utilizar a biblioteca **Mapster**. O *backend* processa estas *mutations* de forma polimórfica e devolve *Output DTOs* desenhados à medida das necessidades da Interface de Utilizador (UI).

7.2 WebSockets (Real-Time Communication)

Empregues para assegurar o contexto colaborativo exigido num sistema de gestão de equipas. A **WebSocketDeliveryStrategy** garante que alterações de estado nas tarefas e alertas de presença sejam refletidos nos quadros virtuais de todos os elementos da equipa de forma instantânea.

8 Gestão do Projeto

- **Controlo de Versões:** Repositório estabelecido em https://github.com/danyraimundo97-bit/pedwm_projeto, com a documentação e visualização de toda a arquitetura
- **Aferição Individual:** As tarefas foram estimadas e monitorizadas utilizando o gestor de tarefas no github <https://github.com/users/danyraimundo97-bit/projects/1>.
- **Modelação Arquitetural:** Devido à extensão da representação da arquitetura do projeto, o diagrama UML completo foi exportado para visualização interativa e em alta resolução, podendo ser consultado através do ficheiro *draw.io* em: https://github.com/danyraimundo97-bit/pedwm_projeto/blob/main/Docs/Diagrama%20de%20classes%20V2.drawio.